补充说明

计算机学院 陈建全
Email: cjq@nwpu.edu.cn
2024/4/21

■不同的工具对类图的表示略有不同。比如，如下代码：

```
package cn.edu.nwpu;
interface Shape  {
        public abstract void Show(); //public
}
class Point implements Shape {
        public  int x,y;
        public  Point(int x, int y){
                this.x=x; this.y =y;
        }
        public void Show() {
                System.out.println("This is point:"+x+","+y);
        }
}
```

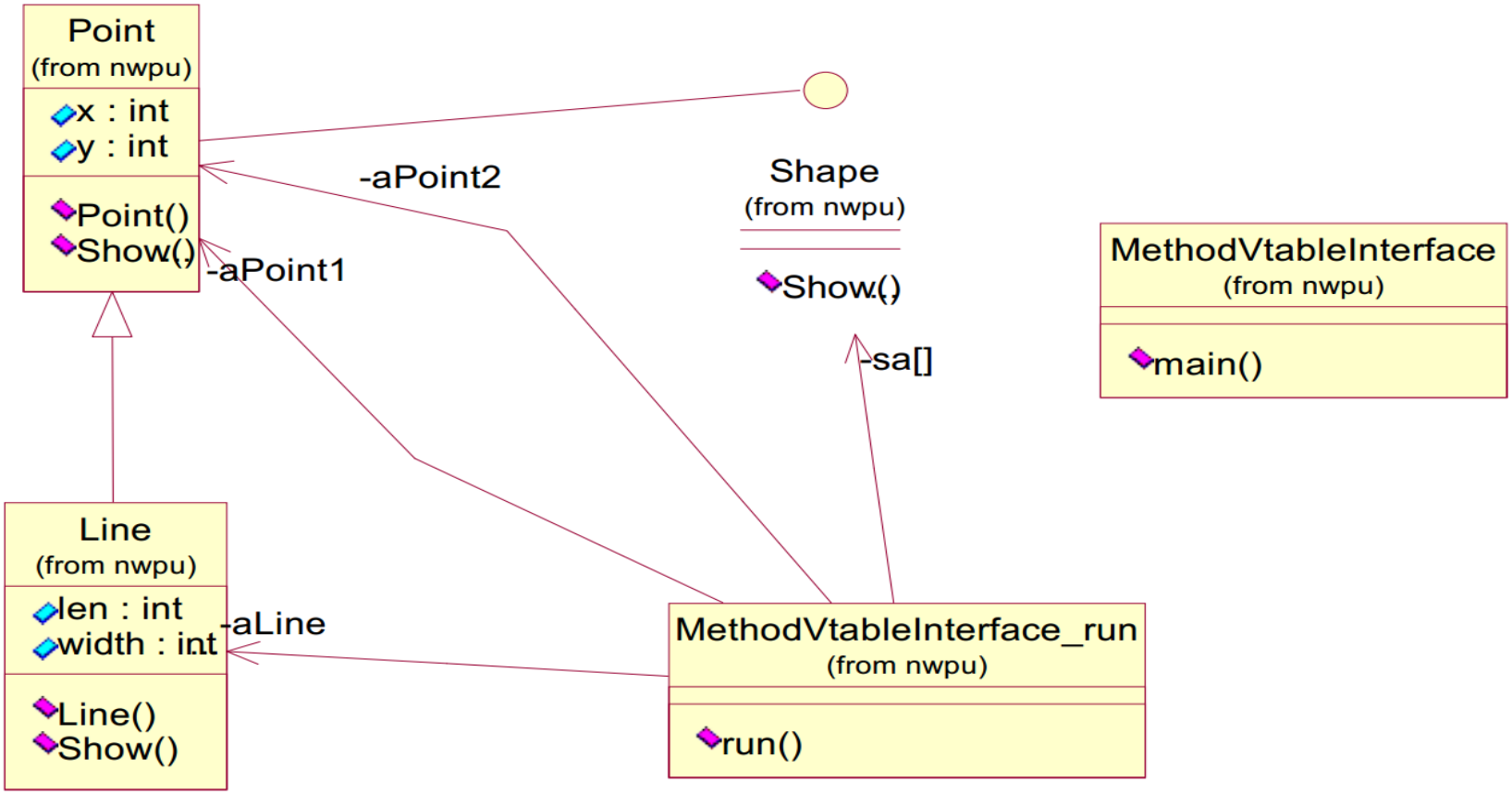
Point类实现Shape接口。Line是Point的子类。

```
class Line extends Point {
        public int len, width;
        public Line(int x, int y, int l, int w)   {
                super(x,y);
                len =l;
                width =w;}
        public void Show(){
                System.out.println("This is line:"+x+","+y+","+len+","+width);
}}
class MethodVtableInterface_run{
        private Shape[] sa = new Shape[3];
        private Line    aLine;
        private Point   aPoint1, aPoint2;
        public void run() {
                sa[0]=  new Point(100,100);
                sa[1] = new Line(100,100, 30,40);
                sa[2] =  new Point(200,200);
                for (Shape s:sa) {
                        s.Show();
                }
}}
```

```
public class MethodVtableInterface{
        public static void main(String[] args)
        {
            MethodVtableInterface_run runit = new MethodVtableInterface_run();
            runit.run();
        }
}
```

Point类实现Shape接口。Line是Point的子类。MethodVtableInterface_run的成员变量类型有Shape, Point, Line类型。

Rose 2007逆向工程产生的类图

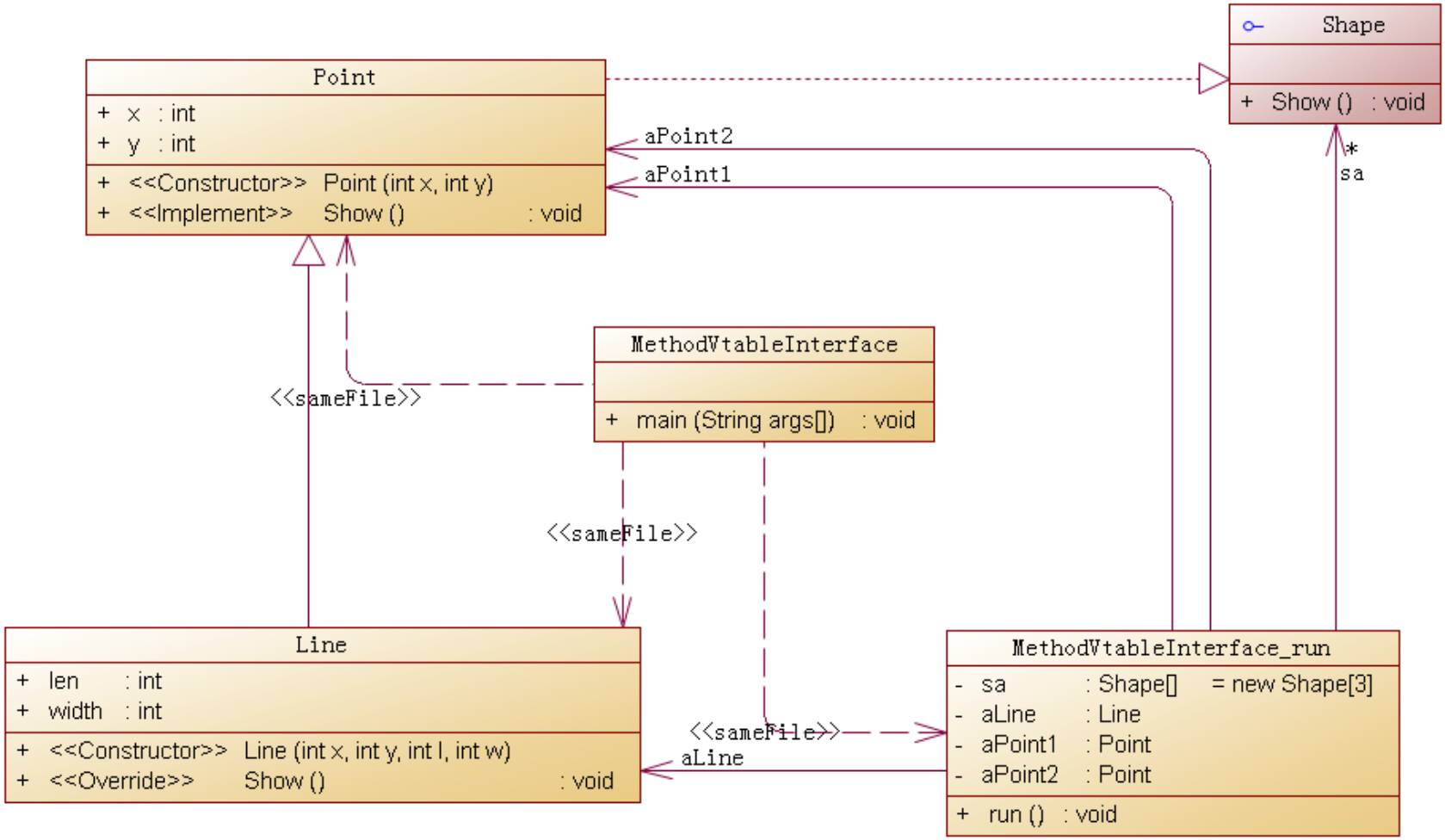


逆向工程生成了类以及关系。然后手动创建类图，把类拖到类图上。其中aPoint1,aPoint2的关联关系重叠，需要手动移动挪开。

点评: MethodVtableInterface的依赖关系没有创建；

利用关联的角色建模成员变量。

PowerDesigner16.6.4.3逆向工程产生的类图

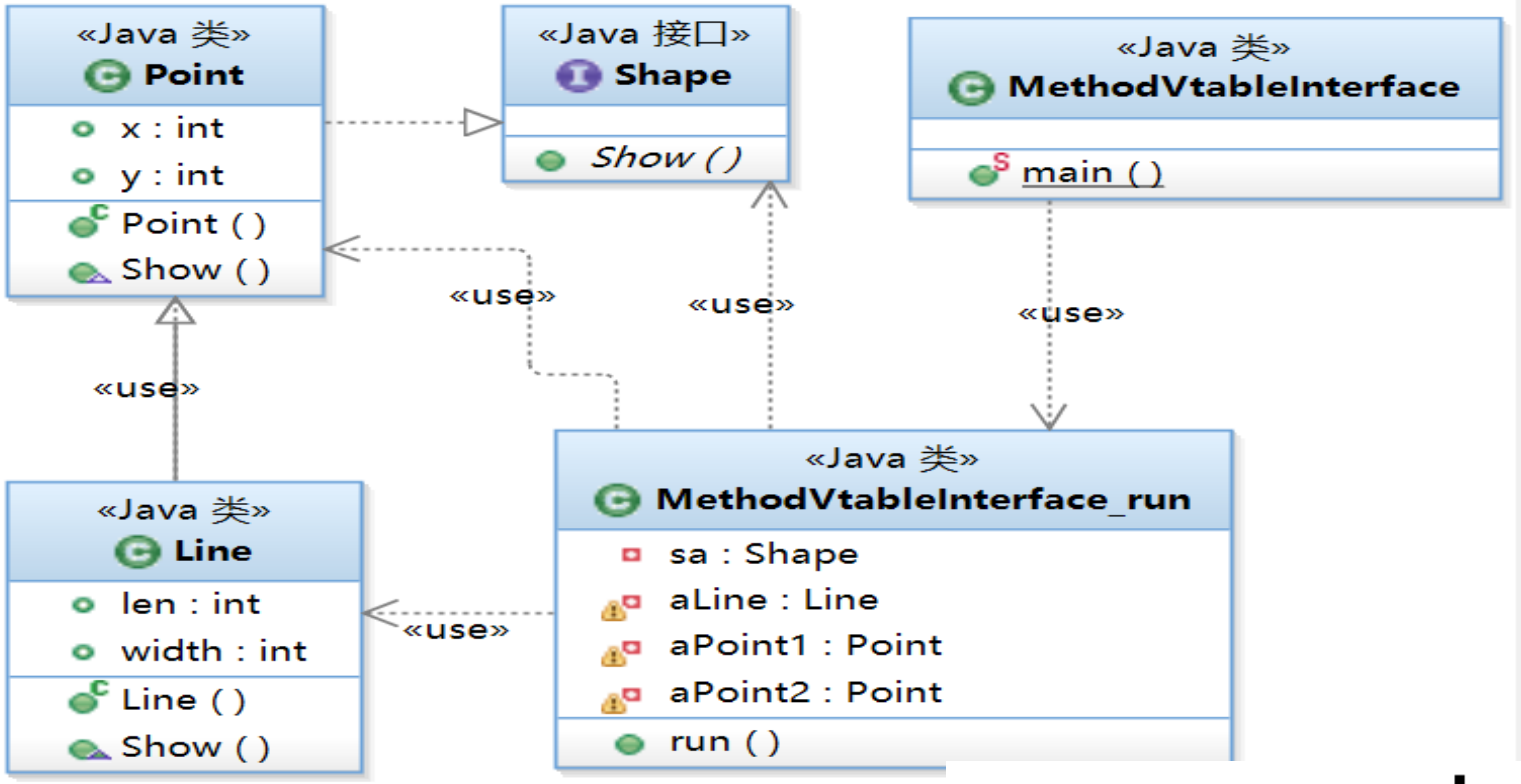


点评: MethodVtableInterface的依赖关系没有必要(sameFile)

成员变量同时标注了关联角色。

利用构造型对方法进行了说明，比较清楚。

RSA 9.0逆向工程产生的类图



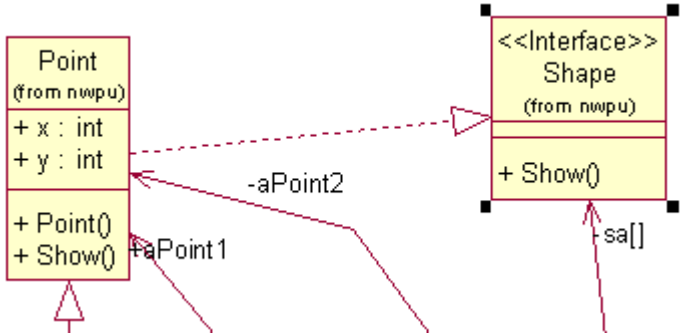
点评: MethodVtableInterface的依赖关系

没有标注关联角色。

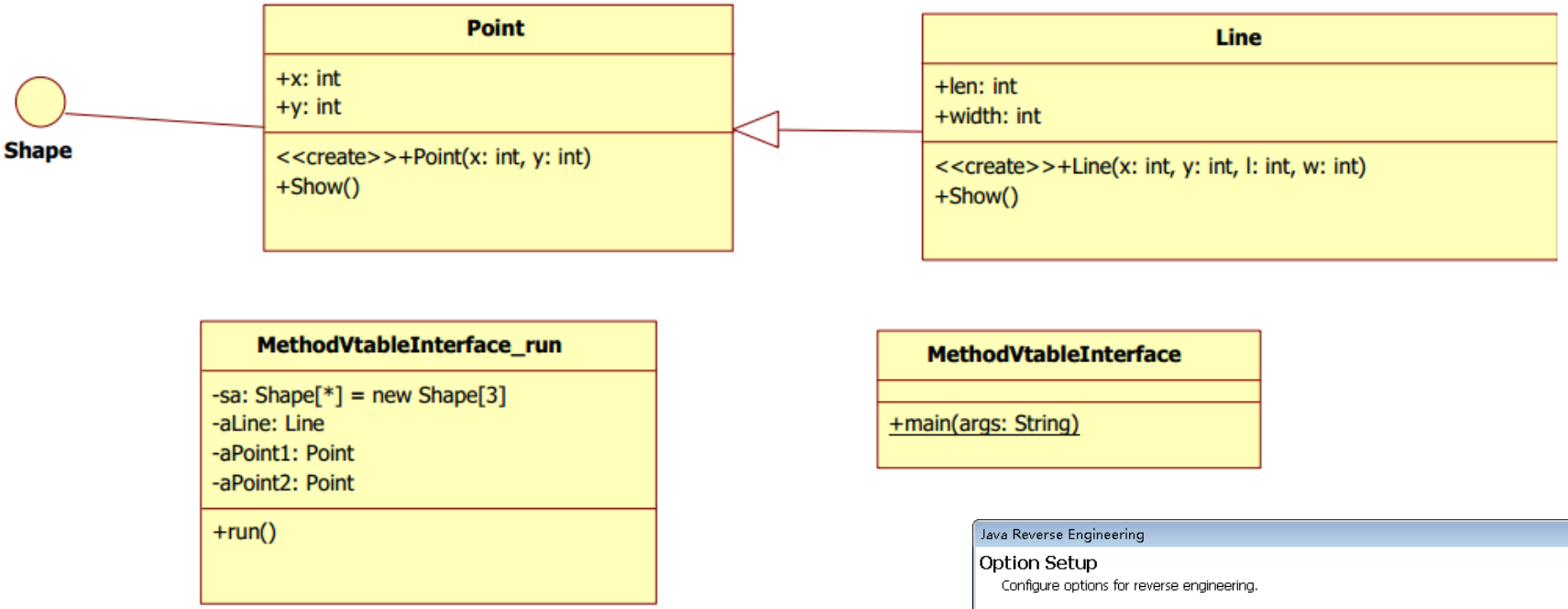
构造方法，覆盖方法利用图标表示

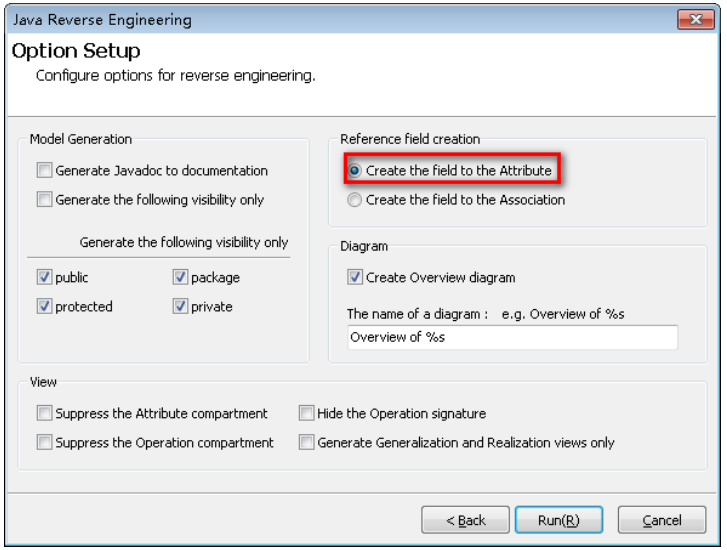
关联关系全部表现为依赖关系。不太好

接口和实现关系的图标



StartUML 5.0.2.1570逆向工程产生的类图



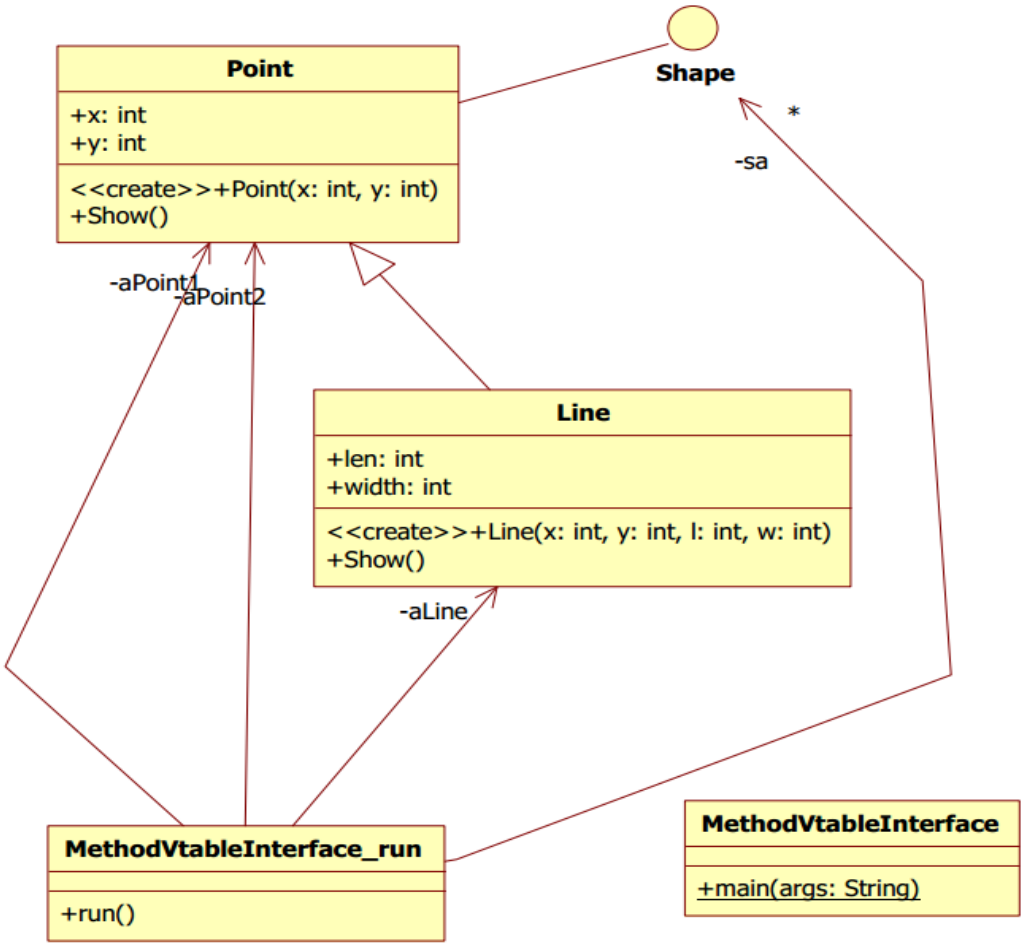


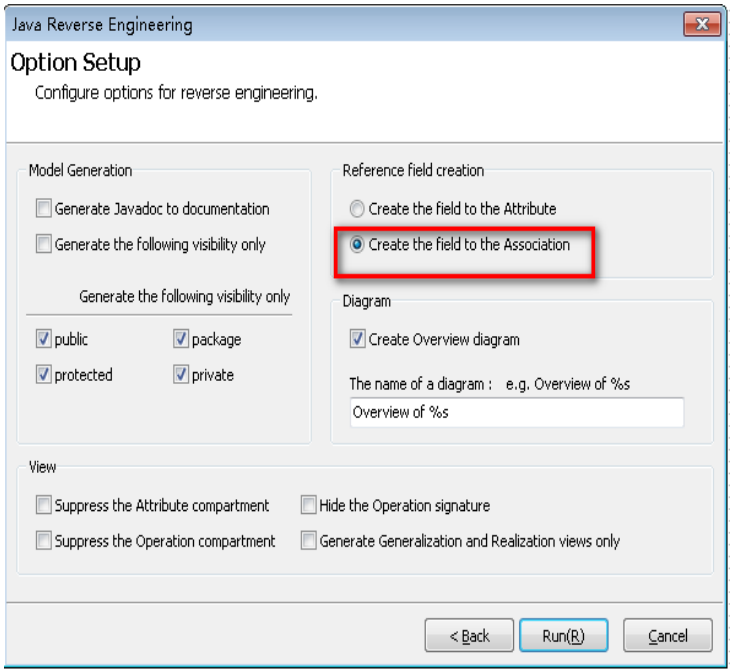
点评: 逆向时候选择了”Create field to the Attribute”

构造方法利用构造型”<<Create>>“标注

依赖关系没有表现

StartUML 5.0.2.1570逆向工程产生的类图





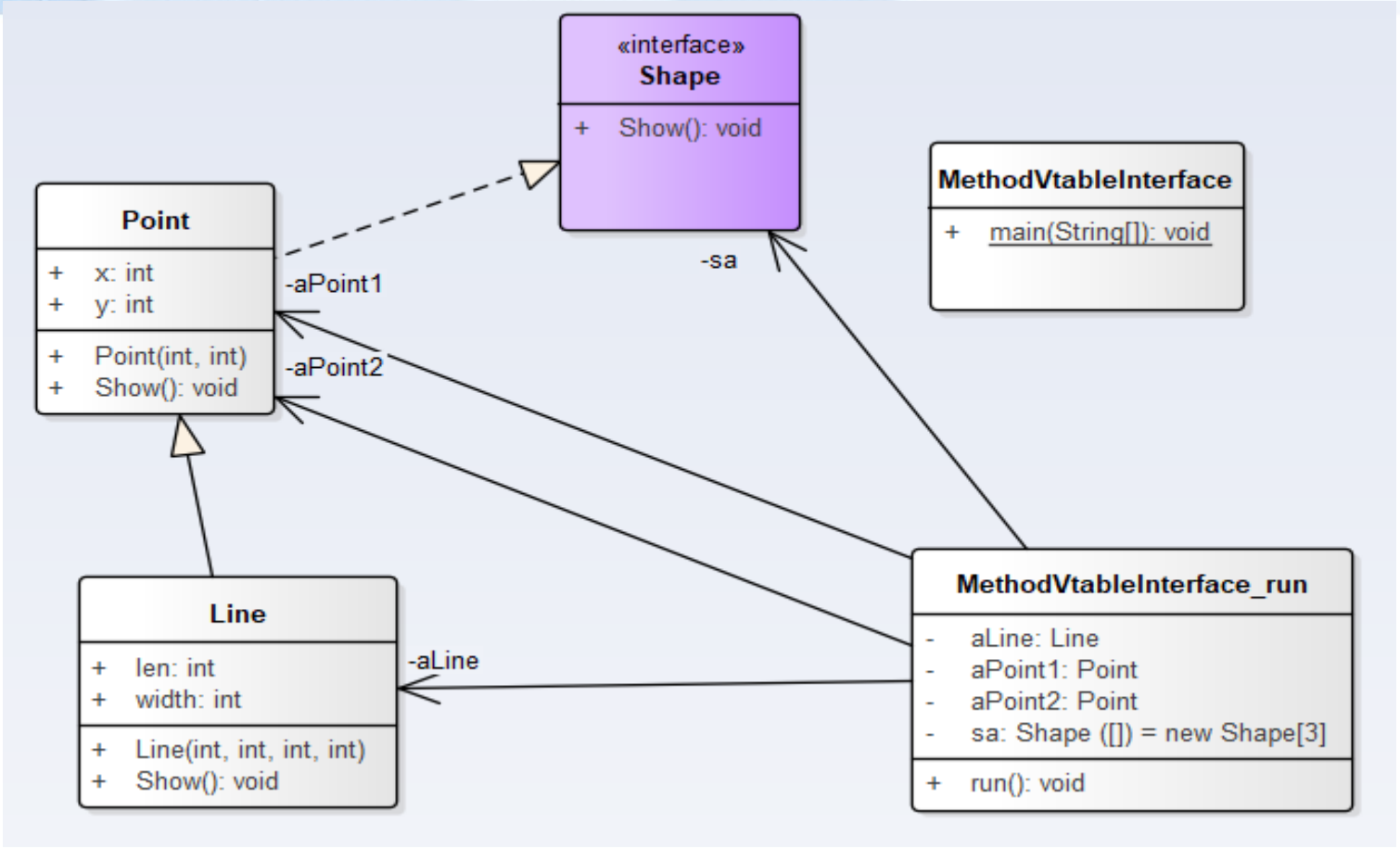
点评: 逆向时候选择了”Create field to the Association”

构造方法利用构造型”<<Create>>“标注

依赖关系没有表现

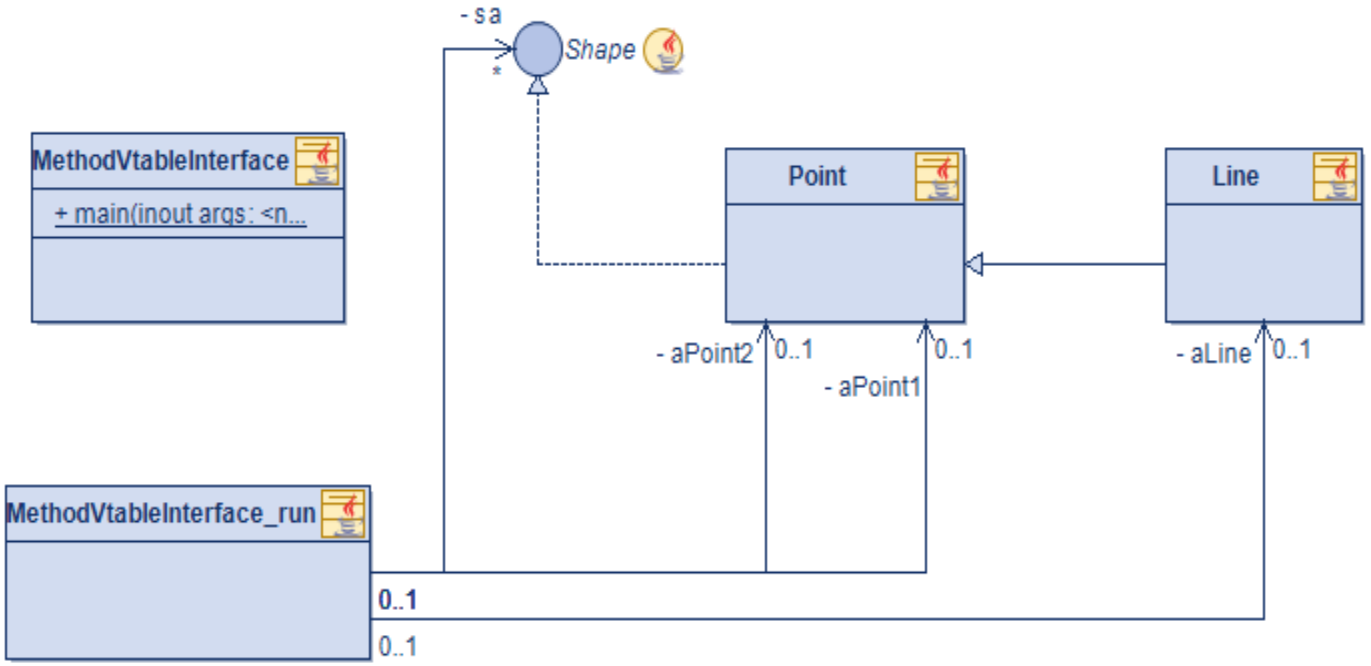
和Rose 2007结果差不多

Enterprise Architec 12逆向工程



点评: MethodVtableInterface的依赖关系没有成员变量同时标注了角色。

Modelio 5.3.1(开源)



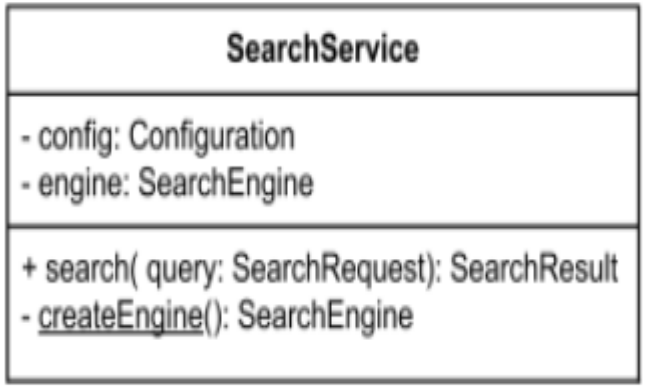
逆向工程生成了类以及关系。然后手动创建类图，把类和关系分别拖到类图上, 手动调整位置。类成员默认不显示，需要的画把类成员拖到类图上。

点评: MethodVtableInterface的依赖关系没有创建;

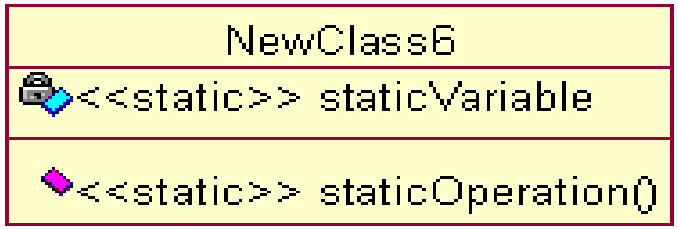
利用关联的角色建模成员变量, 标注了多重度。

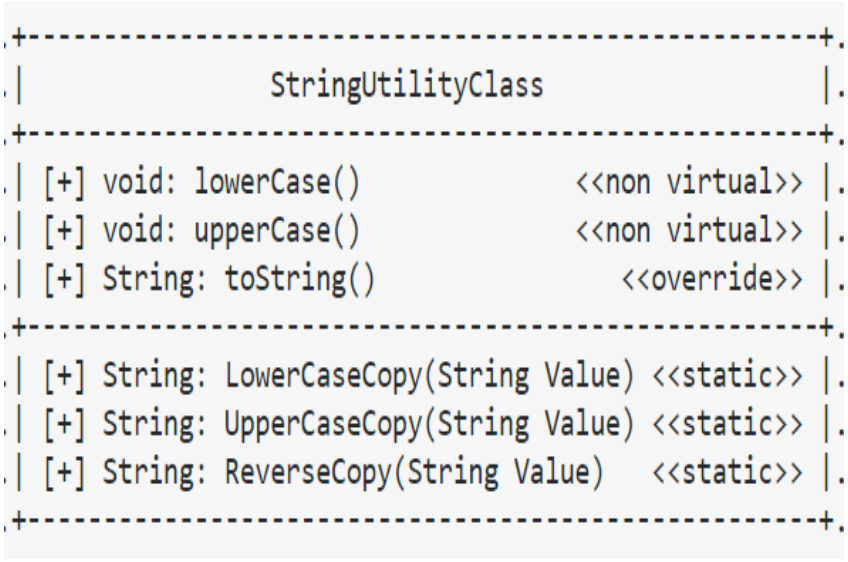
类的静态成员

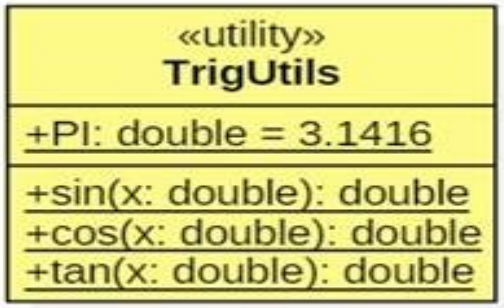
- 一般静态成员不需要明确标注。UML规定静态成员用下划线表示（可以仅把名字用下划线）。实践中也有人利用构造型<<static>>表示静态成员， 或者干脆在旁边加注解。



Class SearchService - implementation level details.
*The createEngine is **static** operation*





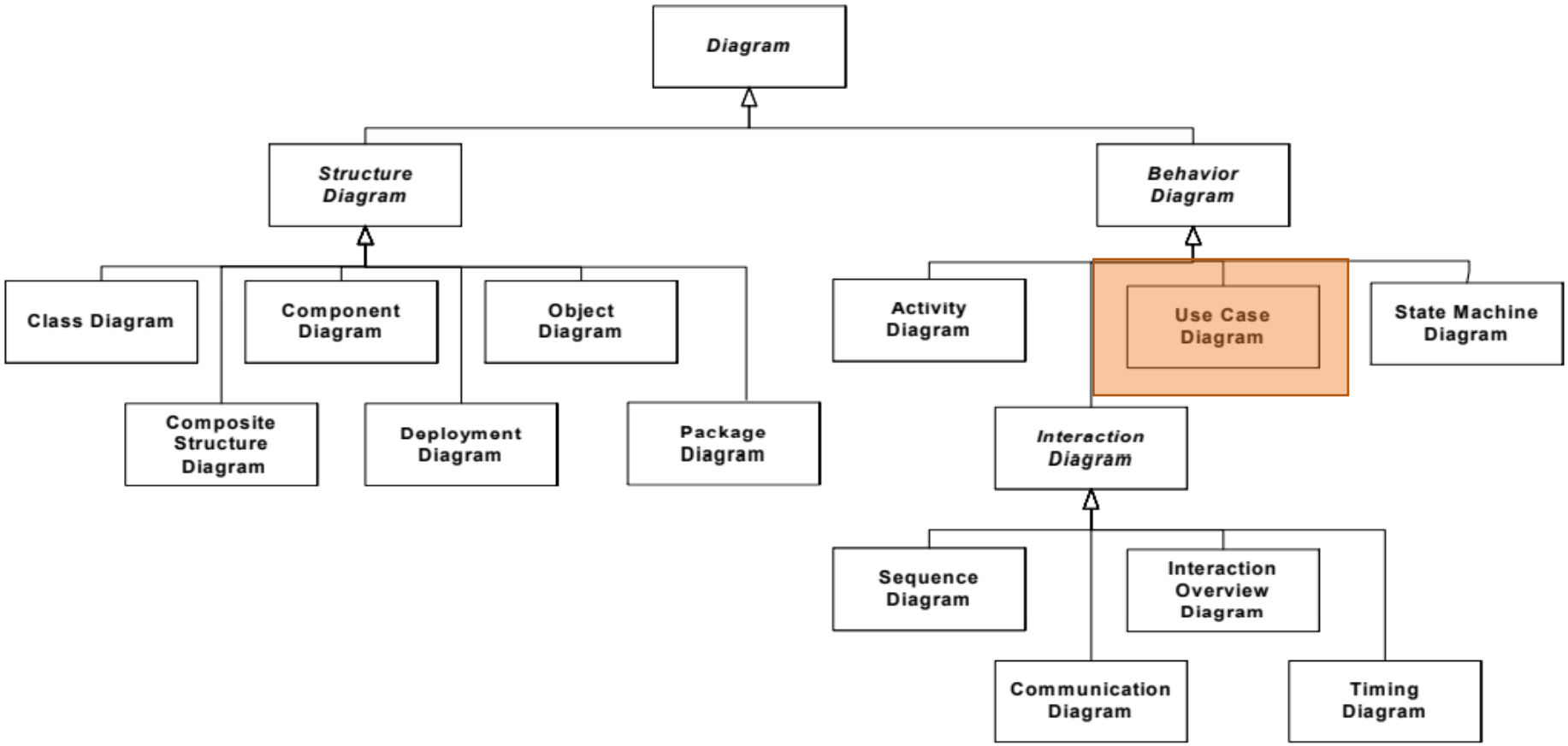


- 规范中的描述是”结构图“和”行为图”
- 为了便于理解记述为“静态结构图”和“动态行为图”
- 注意use-case分类。有些资料单独分类为“用例建模"

UML图的分类

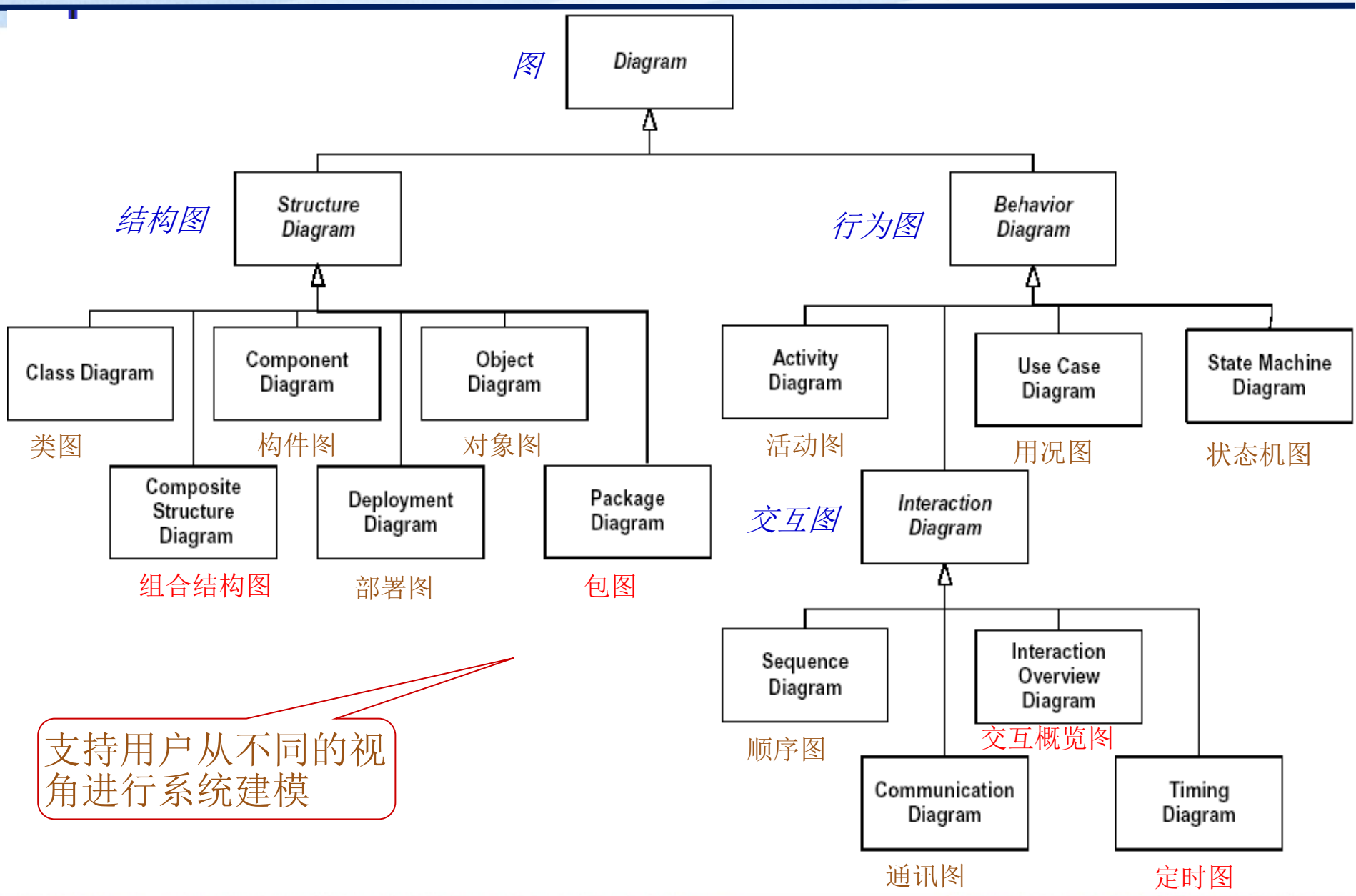
Unified Modeling Language: Superstructure

version 2.0
formal/05-07-04



- BasicBehaviors The BasicBehaviors subpackage of the Common Behavior package introduces the framework that will be used to specify behaviors. The concrete subtypes of Behavior will provide different mechanisms to specify behaviors. A variety of specification mechanisms are supported by the UML, such as automata (“StateMachine (from BehaviorStateMachines)” on page 545), Petri-net like graphs (“Activity (from BasicActivities, CompleteActivities, FundamentalActivities, StructuredActivities)” on page 306), informal descriptions (“UseCase (from UseCases)” on page 578), or partially-ordered sequences of event occurrences (“Interaction (from BasicInteraction, Fragments)” on page 467). Profiles may introduce additional styles of behavioral specification. The styles of behavioral specification differ in their expressive power and domain of applicability. Further, they may specify behaviors either explicitly, by describing the observable event occurrences resulting from the execution of the behavior, or implicitly, by describing a machine that would induce these events.

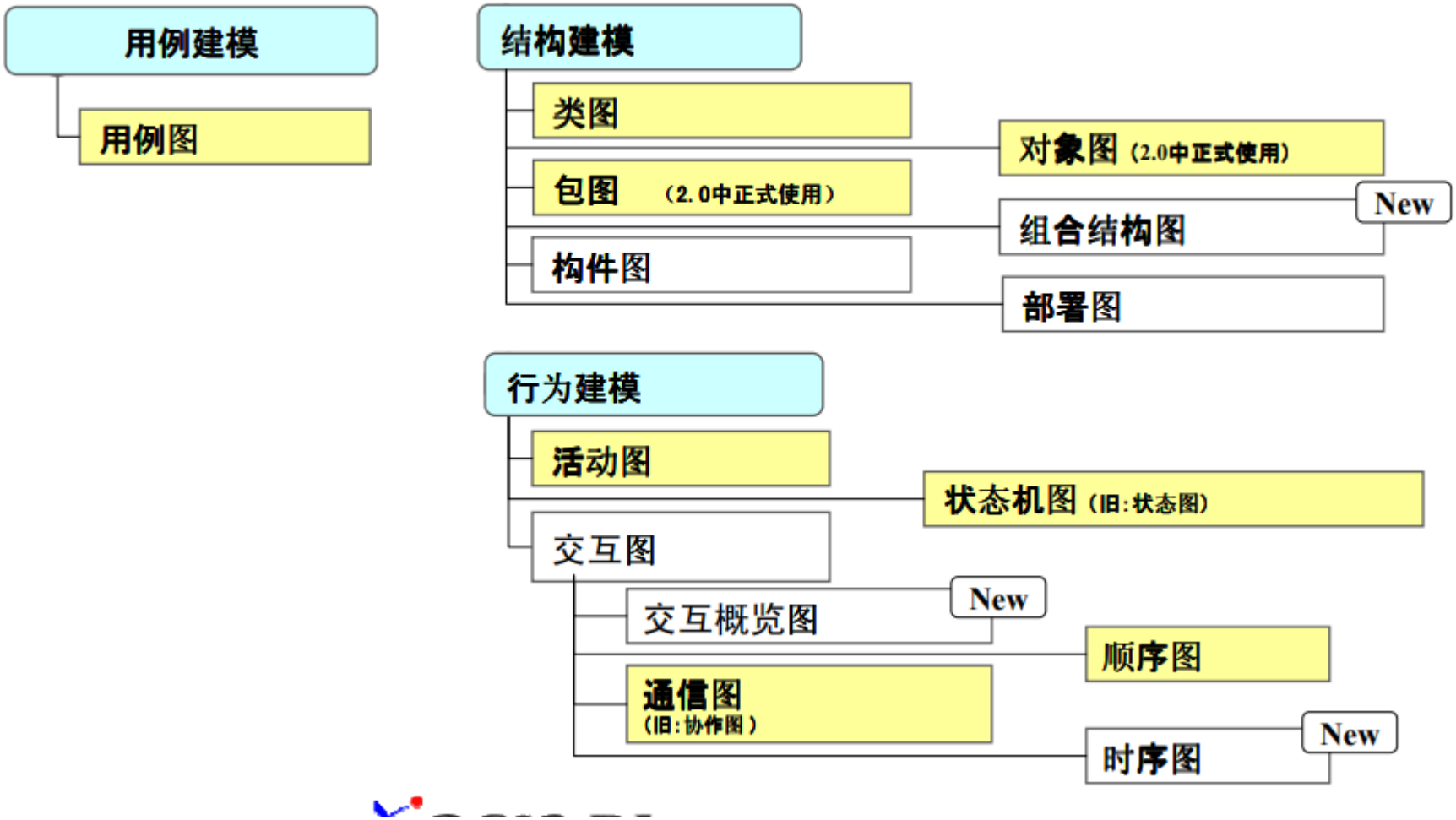
UML2中的各种图



支持用户从不同的视角进行系统建模

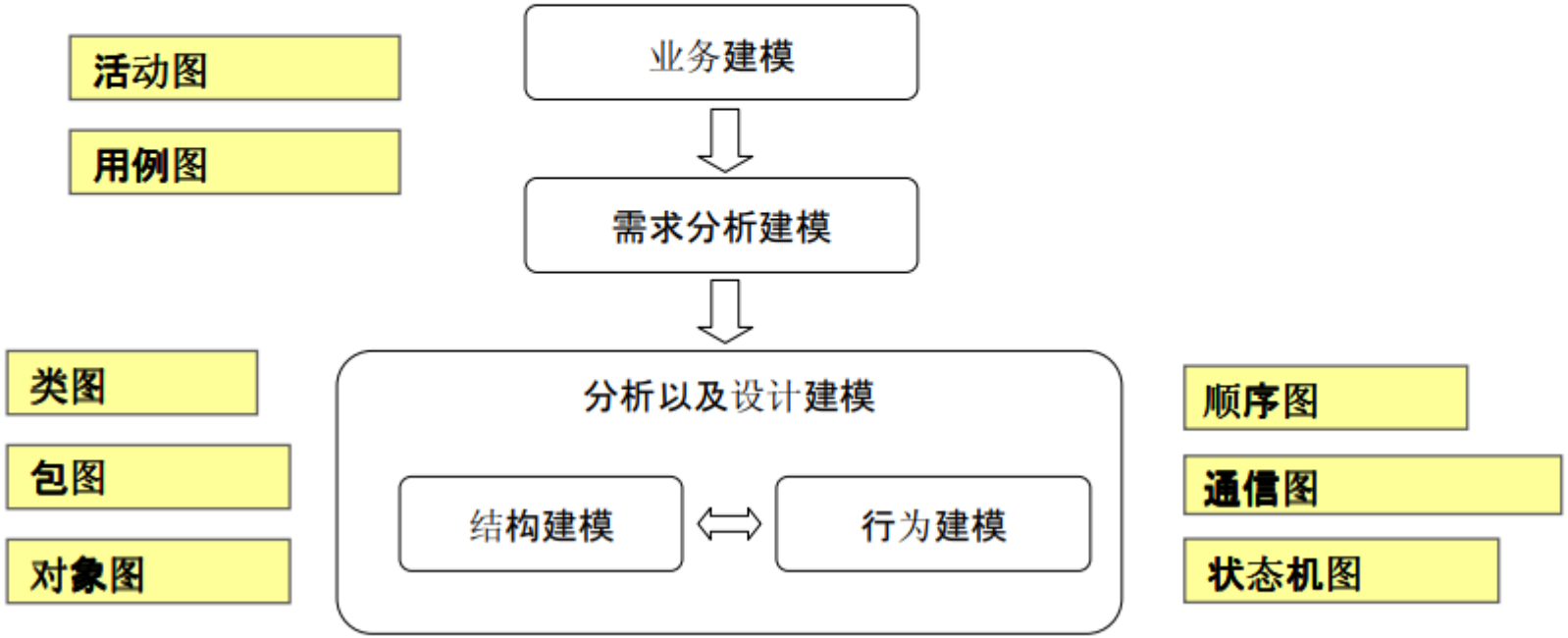
UMTP 2.0的分类

- 建模的分类与UML的关系



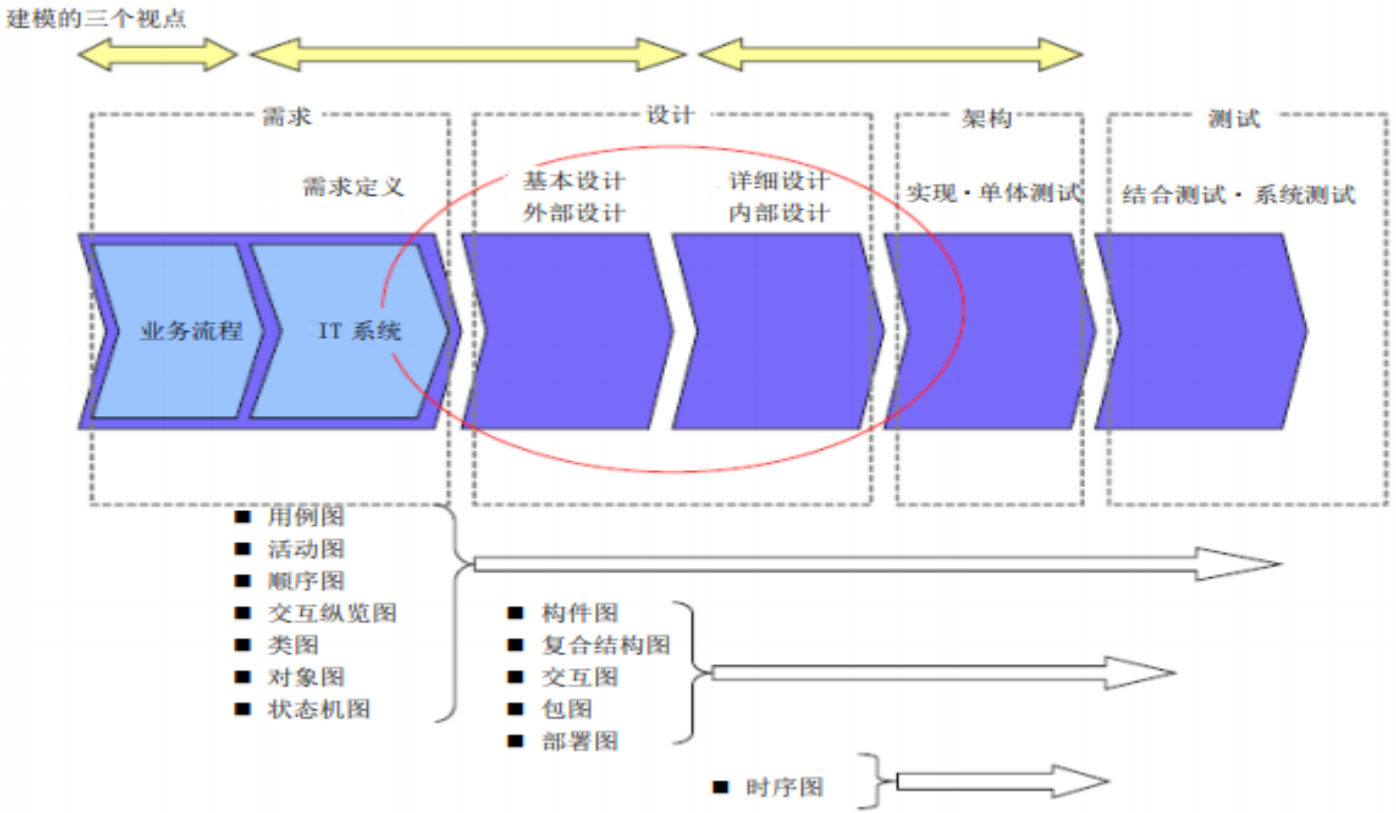
软件开发和UML建模的关系

- 开发阶段与建模以及UML的关系



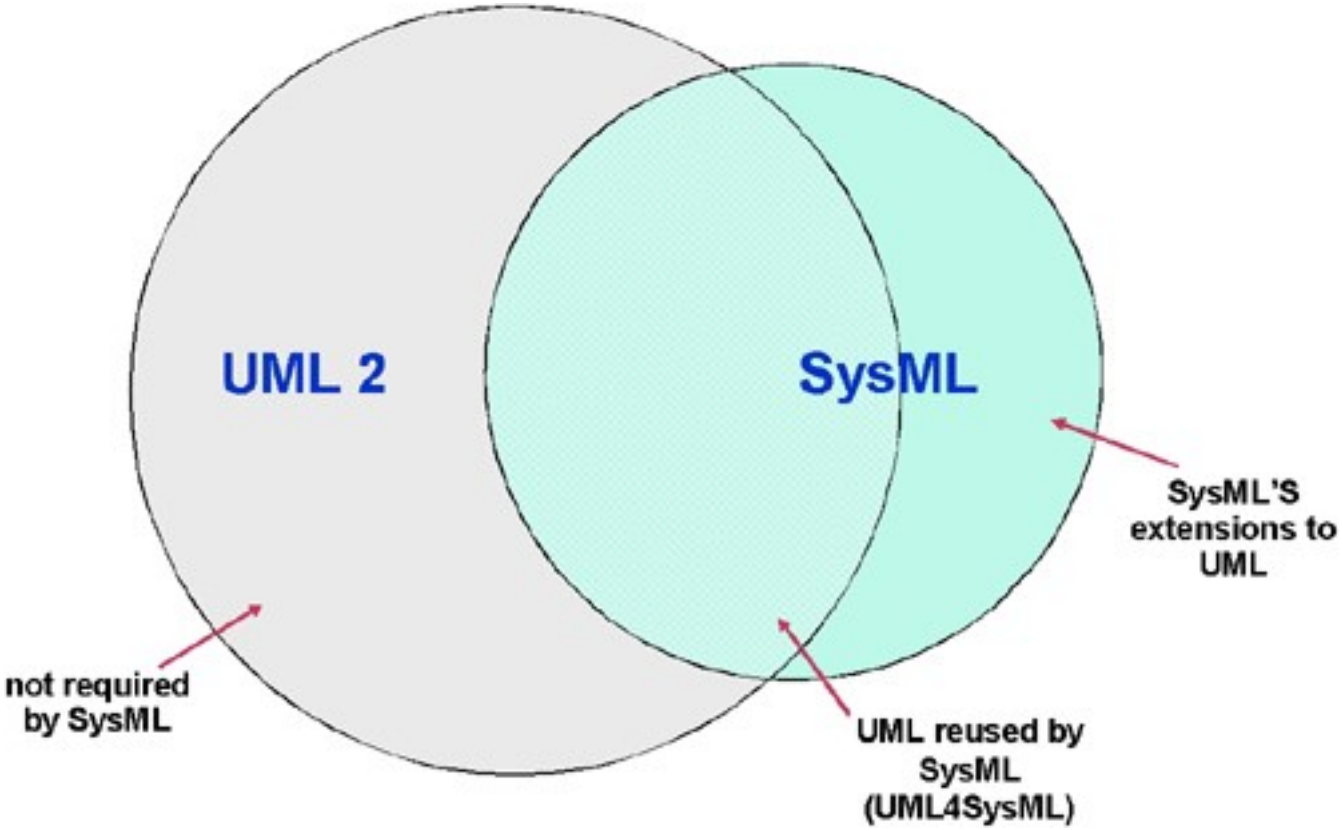
软件开发和UML建模的关系(离岸定制开发)

开发周期从需求阶段后半的需求定义开始、经过设计、直至架构和测试。如果从“UML”的角度出发来看，从实现业务流程的 IT 系统的需求定义开始，到开始编程的架构阶段，都是 UML 的应用对象。具体就是：需求用 UML 来描述、设计式样书用 UML 进行描述。然后把式样传达给离岸开发人员、基于 UML 来进行实现的这样一系列场面。

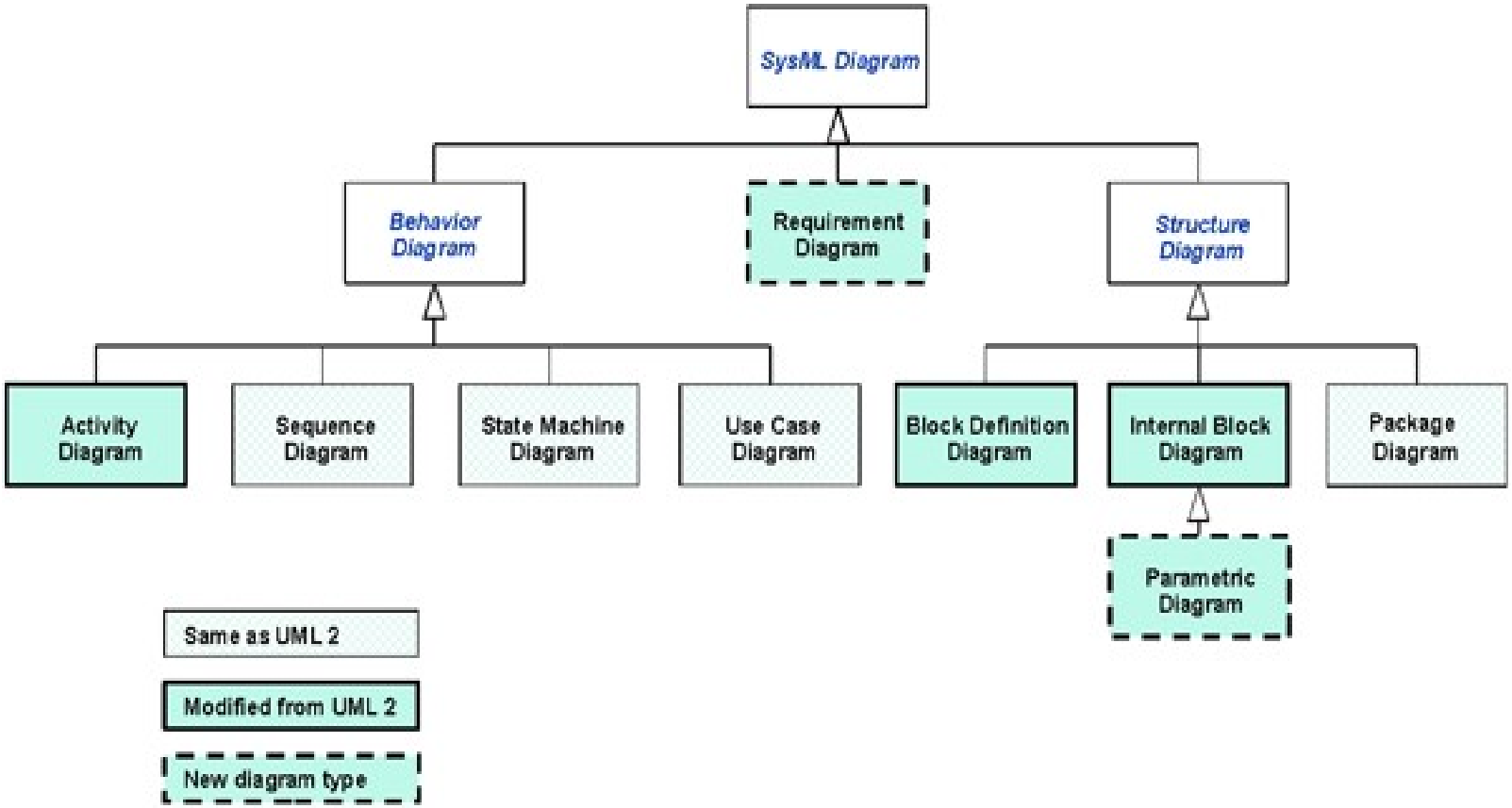


SysML

- OMG系统建模语言OMGSysML®）是一种通用图形建模语言，用于指定，分析，设计和验证可能包括硬件，软件，信息，人员，程序和设施的复杂系统。特别是，该语言提供了图形表示，其具有用于建模系统需求，行为，结构和参数的语义基础，用于与其他工程分析模型集成。
- 它代表了 UML 2的一个子集。并扩展需要满足UML™for Systems Engineering RFP的要求.
- UML™for Systems Engineering RFP由OMG和国际系统工程理事会（INCOSE）共同开发，并由OMG于2003年3月发布.RFP指定了扩展UML以满足系统工程社区需求的要求。该SysML的规格是应通过这些要求开发的不同群体的工具供应商，最终用户，学术界和政府代表。对象管理集团宣布于2006年7月6日通过，并于2007年9月推出OMG SysML™v1.0。


UML 2
SysML
SysML'S extensions to UML
not required by SysML
UML reused by SysML (UML4SysML)

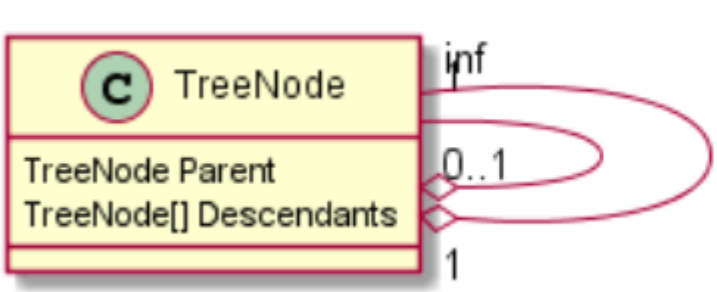
SysML应用现状


SysML Diagram
Behavior Diagram
Requirement Diagram
Structure Diagram
Activity Diagram
Sequence Diagram
State Machine Diagram
Use Case Diagram
Block Definition Diagram
Internal Block Diagram
Package Diagram
Parametric Diagram
Same as UML 2
Modified from UML 2
New diagram type

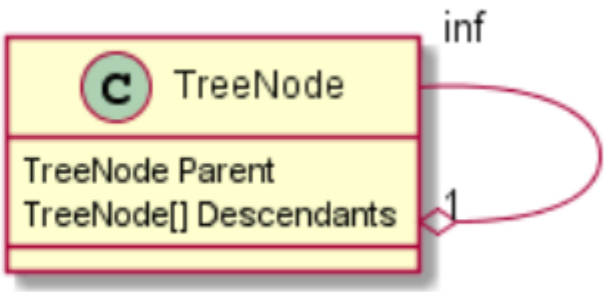
怎样用类图表示树?

```
//pseudocode
class TreeNode {
  TreeNode Parent
  TreeNode[] Descendants
}
```

应该用那个呢?

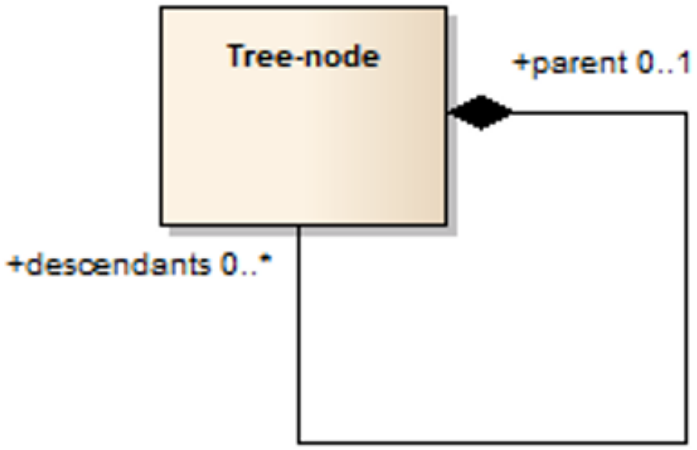


or



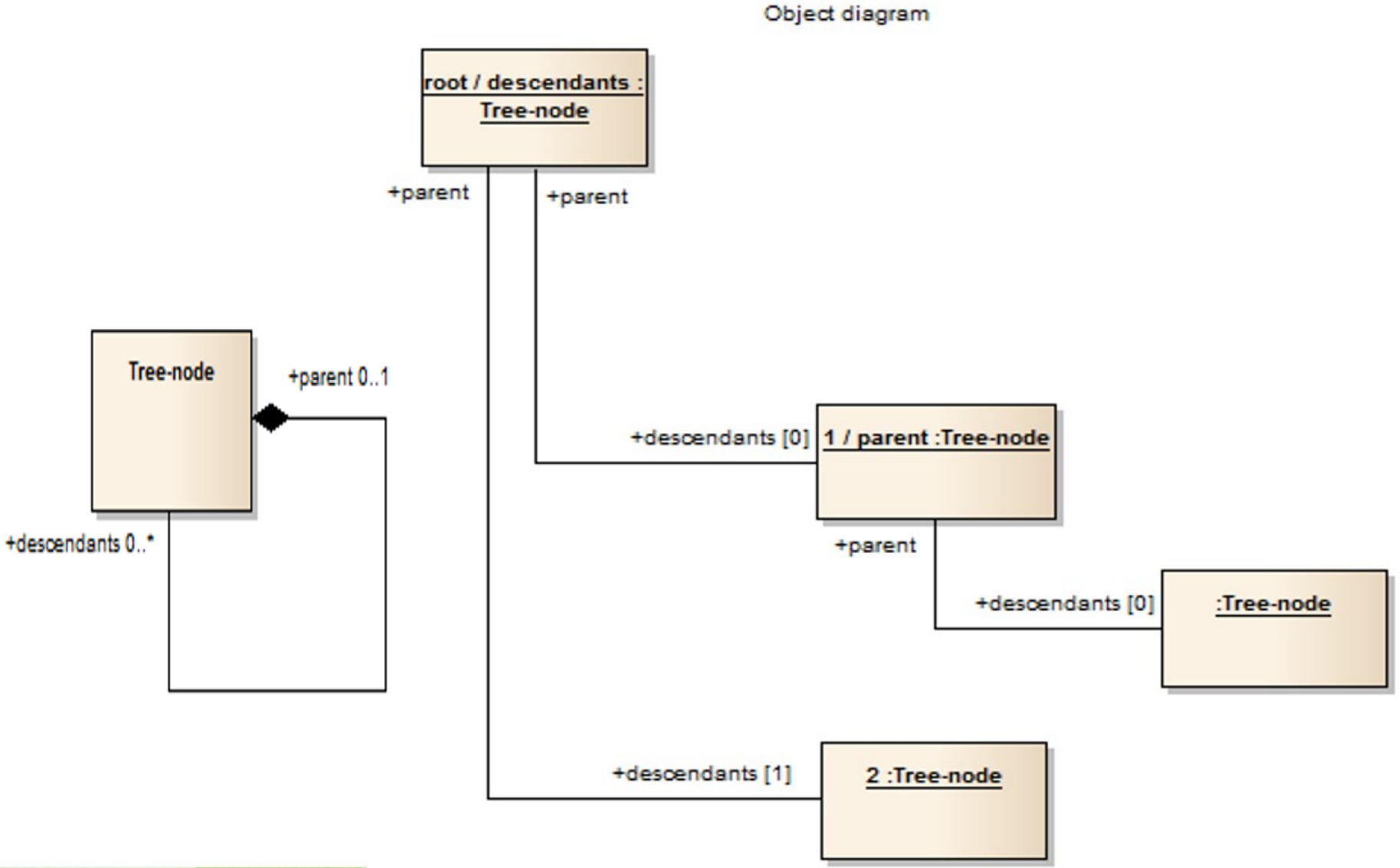
怎样用类图表示树?

- https://stackoverflow.com/questions/24691650/what-is-a-proper-uml-class-diagram-for-a-tree-node-class-with-reference-to-paren
- The trick is to watch **association end roles** (which are simply ignored on your diagram).:
  - "Parent" (0..1), at the association end with the diamond.
  - "Descendants" (0..n) at the opposite association end



怎样用类图表示树?

■可以利用对象图辅助进一步说明。



怎样用类图表示文件和目录?

- For example, consider a file system made of Directories and Files. A Directory can contain any number of FileSystemElements. A FileSystemElement is a concept that denotes either a Directory or a File. In case of a strictly hierarchical system, a FileSystemElement is part of exactly one Directory, which we denote with a one-to-many multiplicity (Figure 2-28).

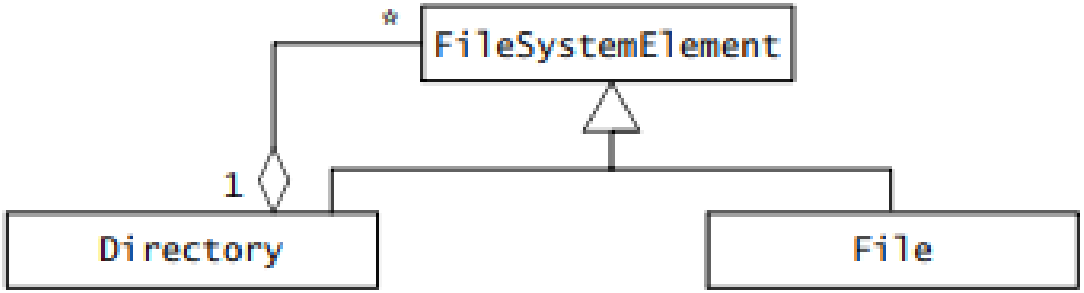
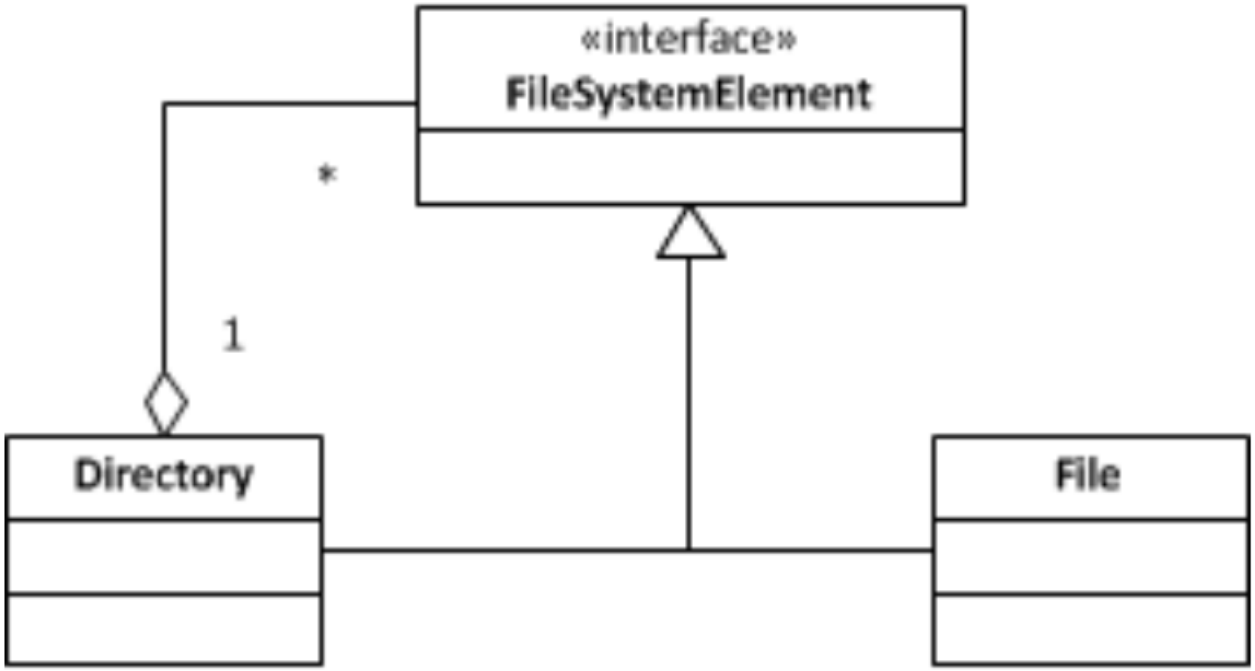


Figure 2-28 Example of a hierarchical file system. A `Directory` can contain any number of *`FileSystemElements`* (a *`FileSystemElement`* is either a `File` or a `Directory`). A given *`FileSystemElement`*, however, is part of exactly one `Directory`.

怎样用类图表示文件和目录?

- 前一页的图是这本书给的:
  - Object-Oriented Software Engineering Using UML, Patterns, and Java （Third Edition），Bernd Bruegge & Allen H. Dutoit
- 有人问为啥不画成这个样子？ (使用接口而不是抽象类)



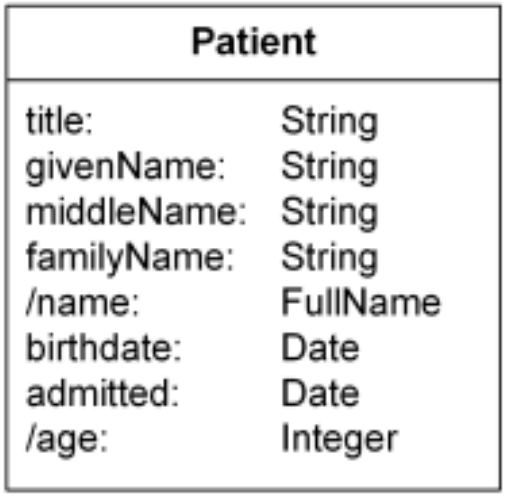
怎样用类图表示文件和目录?

- OOSE 书的作者可能考虑了一些实现策略，因此使用了具体的类。我会做什么取决于这些类的其他用途 - 例如，如果它们只是路径片段的占位符，并依赖其他东西来创建打开文件的流，那么您可以完全测试它们而无需模拟接口，所以我不会为额外的抽象层而烦恼。如果他们有直接的操作来返回流或执行其他文件系统操作，我可能会使用一个接口，以便可以模拟它们进行测试。两者都不正确，或者哪个更合适，无法从两个非常小的 UML 图确定。快速浏览您链接到的 OOSE 书，可以确认它在谈论类图的术语与您使用 ER-Diagrams 的术语大致相同，并考虑表示名词之间的关系，而不是对象的作用。我要提醒的是，如果您正在考虑静态关系中的名词，那么您正在谈论的是过程而不是面向对象的代码 - 也许这就是为什么您会发现书中介绍的模型专注于具有属性的具体类型而不是操作接口。或者它只是试图展示不同类型的关联，不值得深入研究!

- https://softwareengineering.stackexchange.com/questions/146286/how-to-model-a-simple-file-system-by-uml-class-diagram

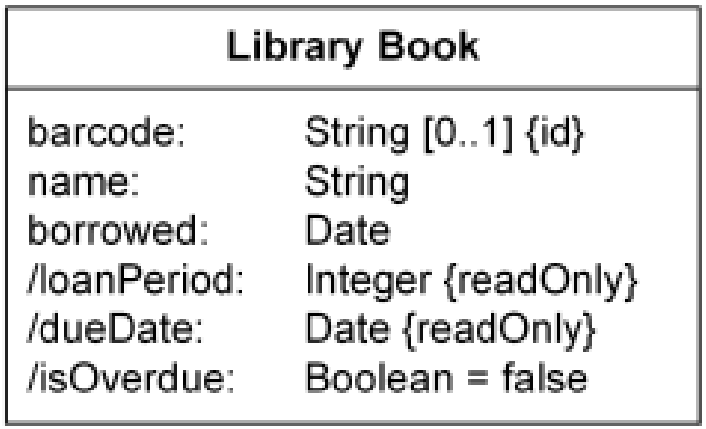
Derived property （派生属性）

- **Derived property** is **property** which value (or values) is produced or computed from other information, for example, by using values of other properties.
- Derived property is shown with its name preceded by a forward slash '/'. This notation is used both for **attributes** and **association ends**.



Derived property （派生属性）

- Because derived property is produced or calculated from other properties, it is common to mark it as **read-only**, so that its value may not be changed directly. UML also allows derived property to be changeable. For the changeable derived property all the constraints have to be met, and in particular the derivation constraint.



UML到底价值如何?

- https://www.quora.com/Is-UML-still-a-valuable-tool-or-just-another-artefact-left-in-the-wake-of-an-industry-that-is-pushing-to-always-go-faster

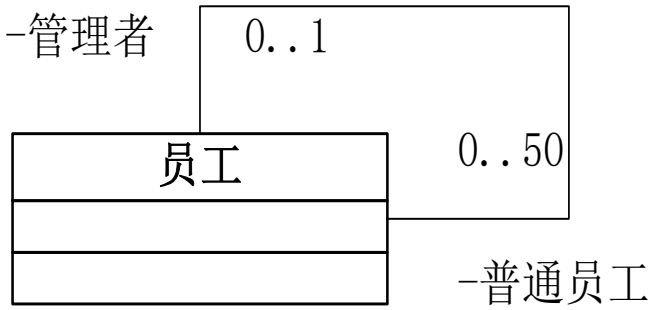
参考网站

- https://www.uml-diagrams.org/about.html
- 作者对OMG组织的规范，进行了更加简单明了的总结。

■ 例题1: 4

◆ 选项2的问题：根据类图，被管理的只有“普通员工”，没有管理者。实际项目中应该对语义进一步明确。

1、一个员工可能管理多个员工，也可能一个员工也不管理√
2、管理者除了能管理普通员工外还可以管理管理者×
3、每个员工都对应一个管理者×
4、一个员工最多可能管理50名普通员工 √

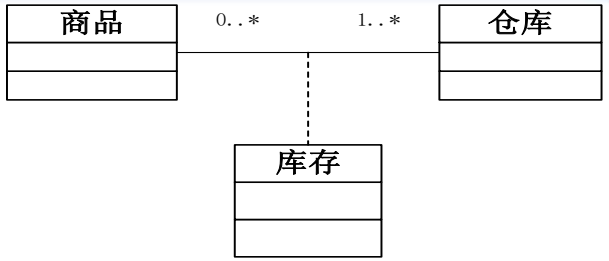


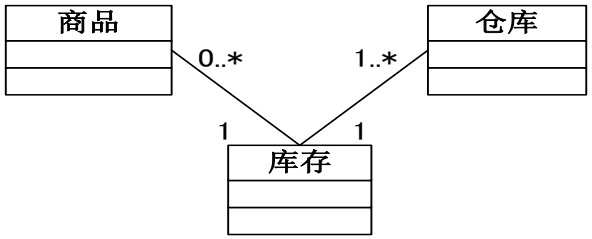
例题1 答案解析

与以下类图等价的类图是 :d

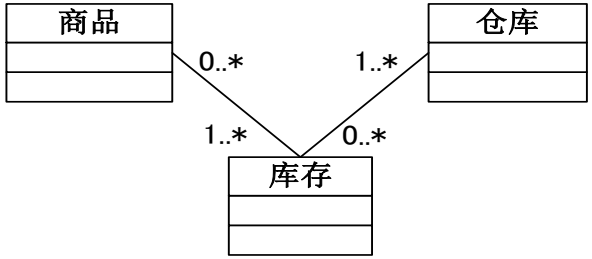
解析： 关联类转换为二元关联，需要“多重度交换”。

读图：从商品看出去，多重度是1..*, 从仓库看出去，多重度是0..*

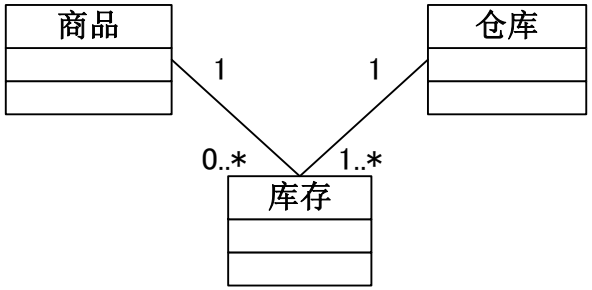




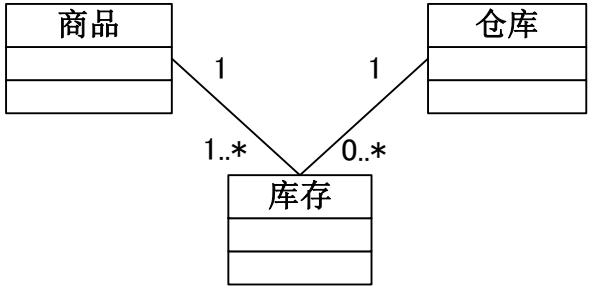
a



b



c

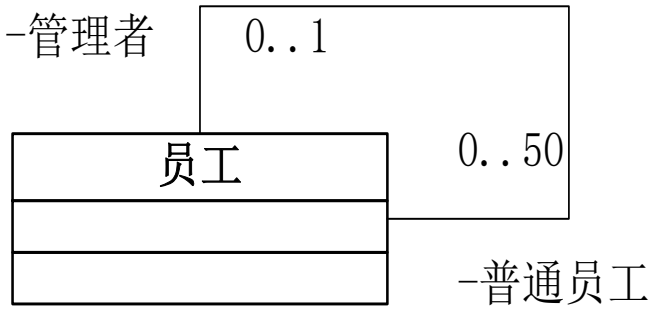


d

■ 例题3:1,4

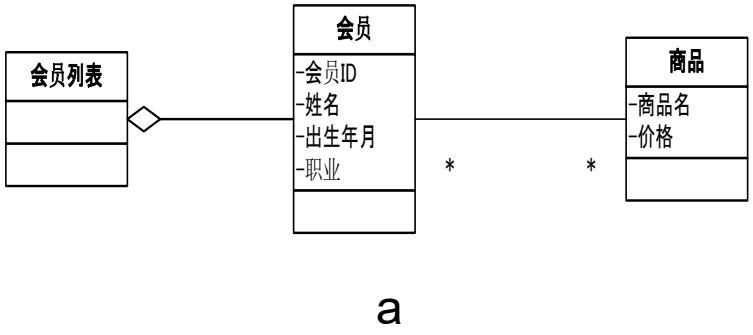
◆ 选项2的问题：根据类图，被管理的只有“普通员工”，没有管理者。实际项目中应该对语义进一步明确。

1、一个员工可能管理多个员工，也可能一个员工也不管理√
2、管理者除了能管理普通员工外还可以管理管理者×
3、每个员工都对应一个管理者×
4、一个员工最多可能管理50名普通员工 √

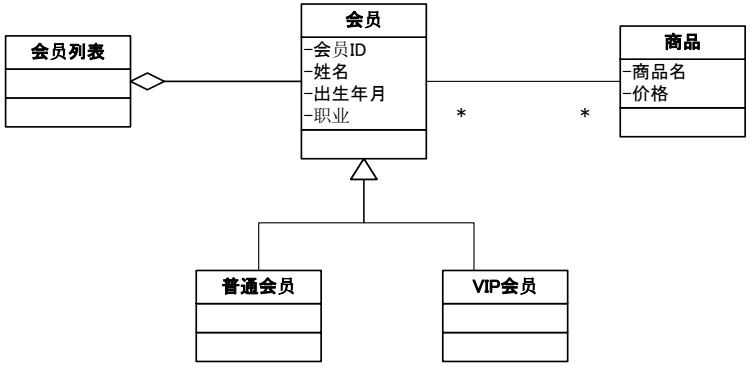


答案解析

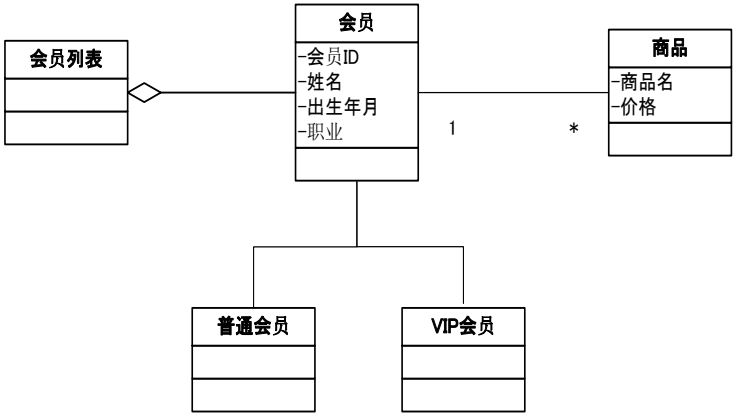
- 例题4:b.　a的问题是没有区分普通和vip回员; c的问题是普通会员和会员应该是泛化关系; d的问题是会员列表和会员之间多重度不对。
- 一般规则：凡是有列表（List)的情况，至少要用聚合。



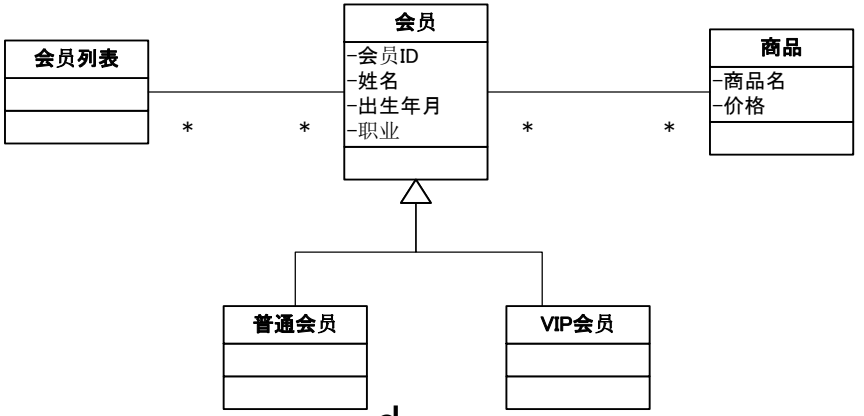
a



b

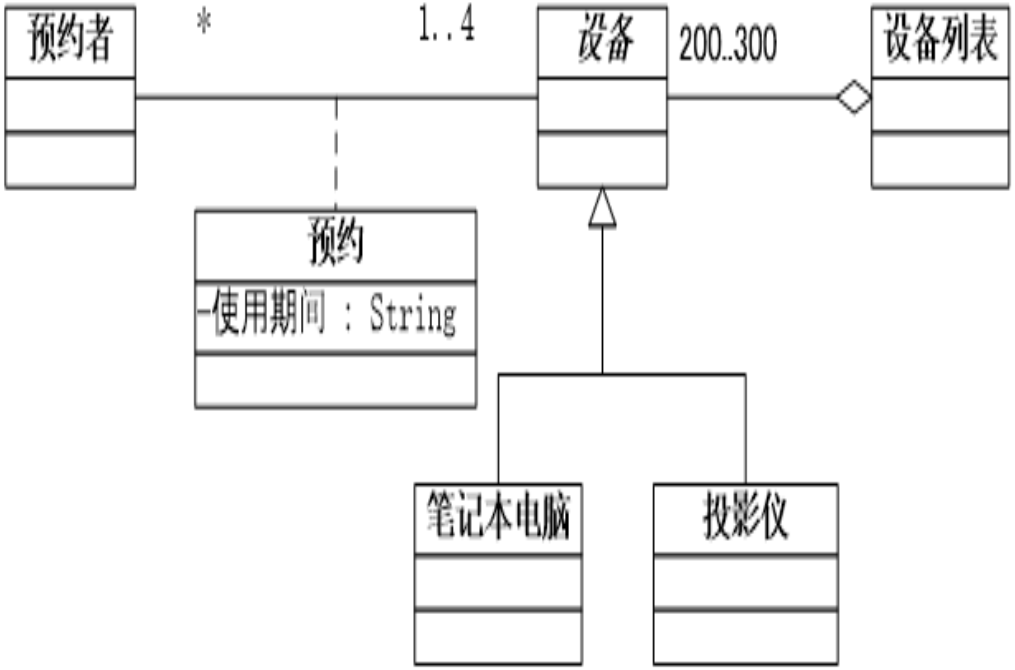


c



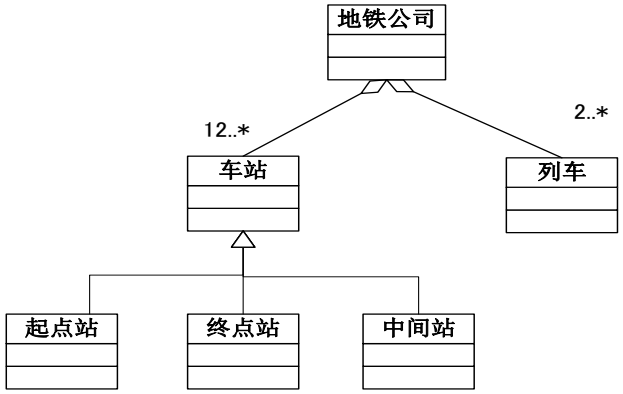
d

- 例题5:
  - 一般规则：凡是有列表（List)的情况，至少要用聚合。
  - 设备列表和设备采用聚合关联
  - 引入关联类，关联类属性按照题目描述。
  - 设备和笔记本、投影机之间的泛化关系。
  - 多重度标注

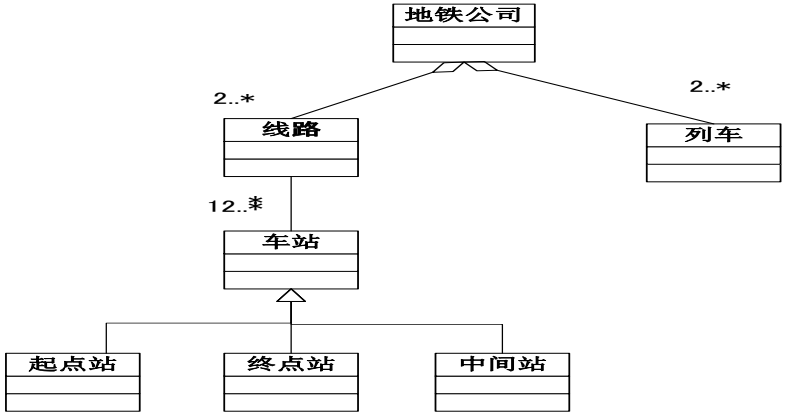


例6. 答案解析

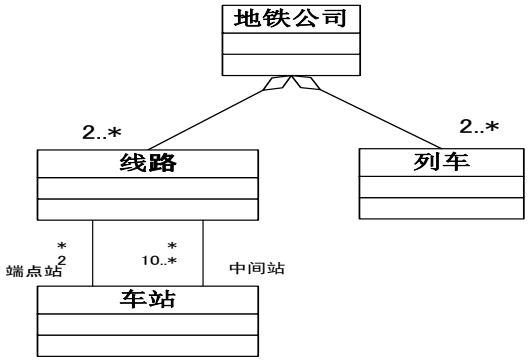
- 例题6: 答案c. a:缺少线路; d: 没有体现车站之间的共享属性; b: 比较接近正确，但是无法区分只有一个起点和终点站。 C: 用角色区分中间站，比b的模型更精确。



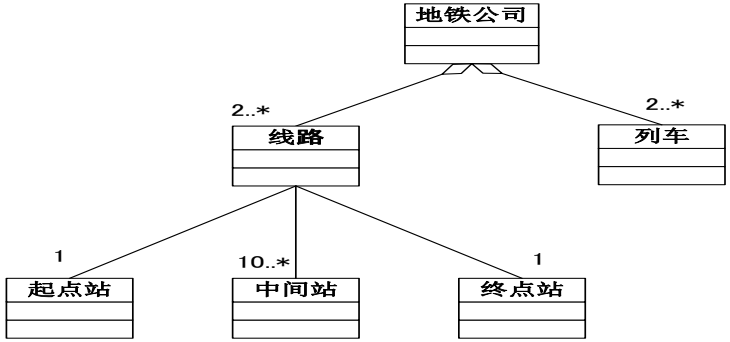
a



b



c

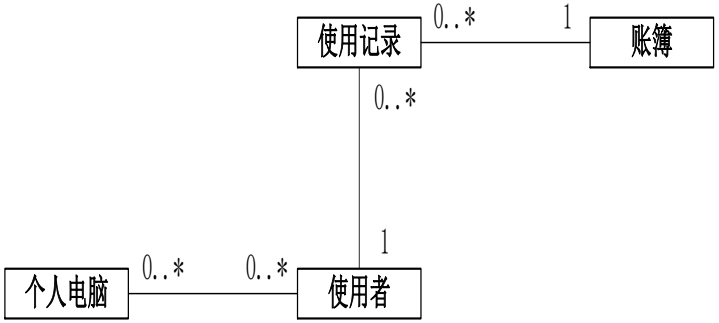


d

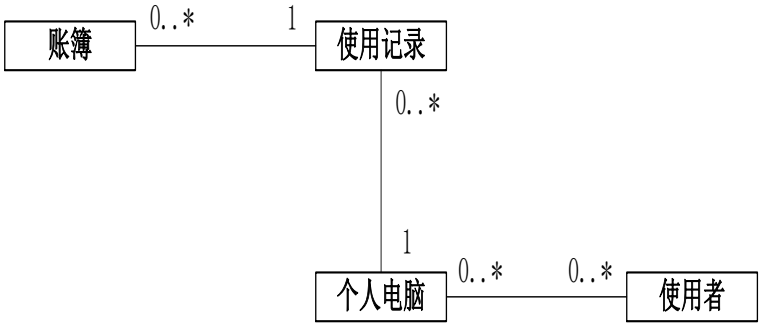
- 例题7: 首先借助员工的泛化关系和关联的名称“评价”，判断出
  - (d) 为(6) 管理层员工
  - (e) 为（5）普通员工

例8. 答案解析

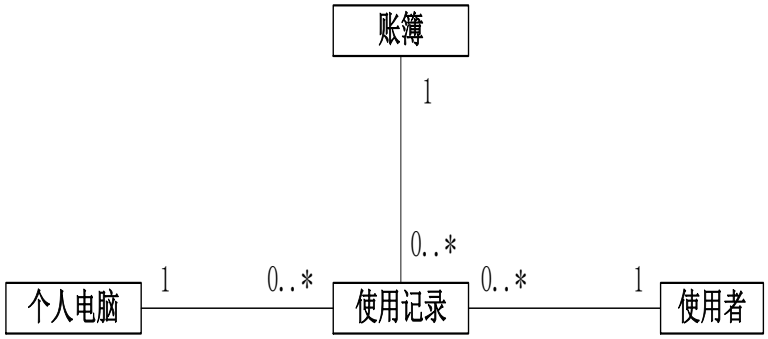
- 答案c. a: 使用记录关联到使用者没有关联到个人电脑，就无法把个人电脑作为关键字查询账簿。b:和a问题类似，无法按照使用者查询账簿。d:比较接近正确。d的不足是无法实现。c比较合理。



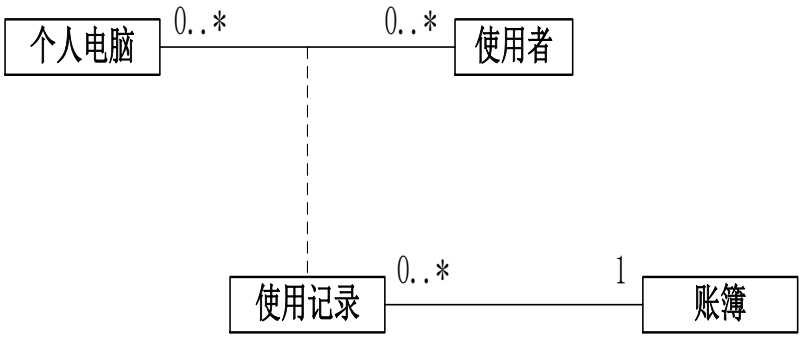
a



b



c



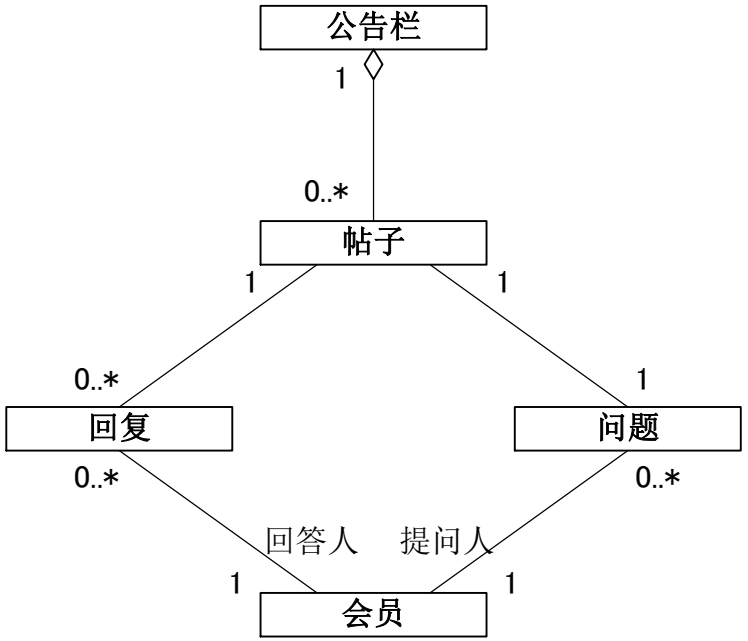
d

例8. 答案解析（了解即可）

- 扩展讨论: 关联两端的实例唯一决定关联类的一个实例。
- 但是UML 2.5.1规范中有如下描述：
  - Even when all ends of the AssociationClass have isUnique=true, it is possible to have several instances associating the same set of instances of the end Classes.
  - isUnique：这是一个UML属性，当设置为true时，意味着关联的每端（即与关联连接的类的每个实例）必须是唯一的。也就是说，对于关联的某一端来说，不会有两个或多个相同实例与另一端的同一个实例关联。
- 理解这个概念，关键是要注意“唯一性”（uniqueness）和“多个关联类实例”之间的区别：
  - 唯一性：假设你有两个类A和B，以及一个将它们联系起来的关联类X。如果关联的每一端isUnique=true，那么对于任意的A和B的实例组合，在关联类X中，这样的组合只能出现一次。也就是说，一个特定的A实例和一个特定的B实例之间的关联在X中是唯一的。
  - 多个关联类实例：尽管A和B之间的关联是唯一的，但可以存在多个X的实例，每个实例都表示A和B之间关系的不同方面或上下文（例如，不同的时间段、地点或条件）。
- "使用者也可以多次使用个人电脑。": 关联只能是唯一的

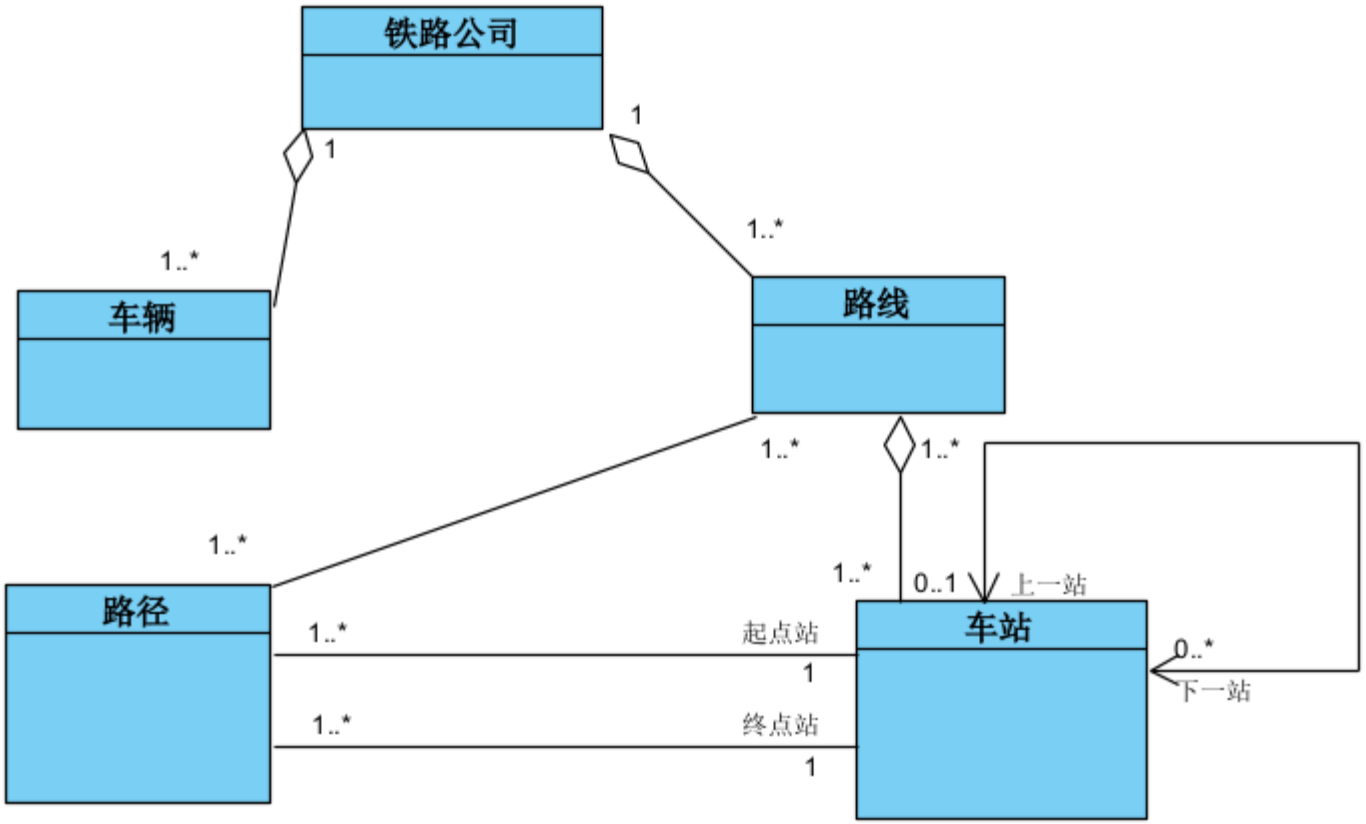
例9. 答案解析

- A．在该模型中，不允许会员回答自己所提问的帖子。（不符合）
- B．在该模型中，也不允许提问的会员回答其它的帖子。（不符合）
- C．在该模型中，会员能对同一帖子回答两次以上。(符合)
- D．在该模型中，一个会员对某帖子只能提问一次。但假如又新创建了帖子，也能提问别的问题。(符合)
- (必要时利用对象图确认)



■a-B b-E c-C d-D e-A

- ◆从角色”起点站，终点站"判定 e是 “车站”
- ◆验证多重性



例11. 答案解析((必要时利用对象图确认)

- A．路径的起点站和终点站有可能属于不同的铁路公司。正确。起点站和终点站可能属于不同公司的不同路线
- B．从某个车站到相邻的车站的多重性应该是[0..*]。 正确,因为车站可能在同一个公司的多个路线上
- C．不同的铁路公司之间也可以共用路线。不正确。路线只能属于一个铁路公司
- D．通过该模型，不知道路线的第一个车站和最后一个车站 不正确。可以判断有无下一站和上一站。

